CSC 5760 – Parallel Programming
Homework #1

Kaelyn
Tennessee Tech University

Due: March 4, 2026

Problem 1: Ring Topology (Hand-Coded)

Approach

Neighbor ranks are computed as:

$$\texttt{next_rank} = (\texttt{rank} + 1) \bmod \texttt{size}$$
$$\texttt{prev_rank} = (\texttt{rank} - 1 + \texttt{size}) \bmod \texttt{size}$$

Deadlock is avoided by having process 0 send before receiving; all other processes receive first, then increment, then send. Every process increments, including process 0 (after receiving at the end of each trip), so the final value after N trips with P processes is $N \times P$. $N = 5$ is hardcoded at compile time.

Code

```cpp
/*
Write an MPI program (by hand, no LLMs) that passes a message of one
integer around in a logical ring of processes with MPI_COMM_WORLD.
The integer should start at 0 in process 0 and be incremented each
time it passes around the ring, and you should be able to have the
message go around the ring N times, where N is specified at compile time.
*/
#include <iostream>
#include <mpi.h>

int main(int argc, char** argv) {

    int N = 5;
    int value = 0;

    MPI_Init(&argc, &argv);

    int rank, size;
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        std::cout << "Need at least two processes for this program."
                  << std::endl;
        return 1;
    }

    int next_rank = (rank + 1) % size;
    int prev_rank = (rank - 1 + size) % size;

    if (rank == 0) {
        value = 0;
        for (int i = 0; i < N; i++) {
            std::cout << "Rank " << rank << " is starting with value "
                      << value << std::endl;
            MPI_Send(&value, 1, MPI_INT, next_rank, 0, MPI_COMM_WORLD);
            MPI_Recv(&value, 1, MPI_INT, prev_rank, MPI_ANY_TAG,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            value++;
        }
        std::cout << "Rank " << rank << " received value " << value
                  << " from process " << prev_rank << std::endl;
    } else {
        for (int i = 0; i < N; i++) {
            MPI_Recv(&value, 1, MPI_INT, prev_rank, MPI_ANY_TAG,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
            value++;
            MPI_Send(&value, 1, MPI_INT, next_rank, 0, MPI_COMM_WORLD);
        }
    }

    MPI_Finalize();
    return 0;
}
```

Results

$N = 5$, $P = 4$. Expected final value: $5 \times 4 = 20$.

Processes (P)	Trips (N)	Final Value
4	5	20

Table 1: Ring correctness verification

Problem 2: Ring Topology — LLM Vibe Coding Comparison

LLM Environment

Attribute	Value
Model	Claude Sonnet 4.6 (`claude-sonnet-4-6`)
Provider	Anthropic
Interface	claude.ai web chat
Session Date	February 18, 2026

Prompts & Responses

Prompt 1:

> *"Generate an MPI program that uses 4 processes in a ring. The program will take in how many times around the ring the starting data (0) will go. Each node the value bounces to will increment it by one."*

Response: Claude produced a working program. It used `scanf` to read the trip count at runtime and distributed it via `MPI_Bcast`.

Prompt 2:

> *"Remind me how to compile and run this."*

Response: Claude gave `mpicc -o ring ring.c` and `mpirun -n 4 ./ring`. The file extension was `.cpp`, causing a linker error with `mpicc`; Claude resolved this by recommending `mpicxx`.

Prompt 3:

> *"I think it's stuck in a loop. It runs and nothing happens. Add output to the program so I can see what it's doing."*

Response: Claude added `fflush(stdout)` and per-process diagnostics. The hang was `scanf` waiting on stdin, which `mpirun` does not reliably forward. After entering the trip count manually, it ran correctly:

```
Final value after 4 trips: 12
```

The LLM-generated code (with debug output from Prompt 3) is shown below.

```c
#include <mpi.h>
#include <stdio.h>

#define NUM_PROCS 4

int main(int argc, char *argv[]) {
    int rank, size, value = 0;
    int right, left;
    int trips;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```c
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Process %d initialized\n", rank);
    fflush(stdout);

    if (size != NUM_PROCS) {
        if (rank == 0) printf("Please run with %d processes.\n", NUM_PROCS);
        MPI_Finalize();
        return 1;
    }

    right = (rank + 1) % size;
    left  = (rank - 1 + size) % size;

    printf("Process %d: left neighbor = %d, right neighbor = %d\n",
           rank, left, right);
    fflush(stdout);

    if (rank == 0) {
        printf("How many times around the ring? ");
        fflush(stdout);
        scanf("%d", &trips);
        printf("Process 0: will do %d trips\n", trips);
        fflush(stdout);
    }

    MPI_Bcast(&trips, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d: received trips = %d via Bcast\n", rank, trips);
    fflush(stdout);

    for (int t = 0; t < trips; t++) {
        if (rank == 0) {
            printf("Process 0: starting trip %d, sending value %d "
                   "to process %d\n", t, value, right);
            fflush(stdout);
            MPI_Send(&value, 1, MPI_INT, right, 0, MPI_COMM_WORLD);
            printf("Process 0: waiting to receive from process %d\n", left);
            fflush(stdout);
            MPI_Recv(&value, 1, MPI_INT, left, 0, MPI_COMM_WORLD,
                     MPI_STATUS_IGNORE);
            printf("Process 0: completed trip %d, received value %d\n",
                   t, value);
            fflush(stdout);
        } else {
            printf("Process %d: trip %d, waiting to receive from "
                   "process %d\n", rank, t, left);
            fflush(stdout);
            MPI_Recv(&value, 1, MPI_INT, left, 0, MPI_COMM_WORLD,
                     MPI_STATUS_IGNORE);
            value++;
            printf("Process %d: trip %d, received and incremented to "
                   "%d, sending to process %d\n", rank, t, value, right);
```

```
            fflush(stdout);
            MPI_Send(&value, 1, MPI_INT, right, 0, MPI_COMM_WORLD);
        }
    }

    if (rank == 0) {
        printf("Final value after %d trips: %d\n", trips, value);
    }

    MPI_Finalize();
    return 0;
}
```

Comparison & Analysis

Both versions correctly implement ring communication and deadlock avoidance. Two differences:

- **Increment logic:** The hand-coded version increments on every process including rank 0, giving a final value of $N \times P = 20$. The LLM only increments on ranks 1–3, giving 12. The spec is ambiguous on this point.
- **Compile-time N:** The spec requires N at compile time. The LLM used `scanf`, which hung under `mpirun` and required an extra debug prompt to resolve.

Problem 3: Dot Product with Tree Reduction (Hand-Coded)

Approach

Each process receives `N/P` elements starting at `rank * (N/P)` and computes a local partial sum. Partials are combined via binary tree reduction: each round, senders transmit to a partner and exit; receivers accumulate and continue. `step` doubles each round until rank 0 holds the total. Both vectors are all 1s, so the expected result is N. `MPI_Wtime()` times the computation and reduction together.

Code

```
/*
Problem 3 (15pts/12pts):
Write a parallel vector-vector element by element multiplication
program (by hand, no LLM) followed by a tree reduction to compute
the dot-product of two vectors of length N with P processes using MPI.
*/
#include <iostream>
#include <mpi.h>
#include <vector>

using namespace std;

int scalar_product(vector<int> a, vector<int> b, int start, int end) {
    int product = 0;
    for (int i = start; i < end; i++)
        product += (a[i] * b[i]);
```

```
    return product;
}

int main(int argc, char** argv) {

    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int N = 4096;
    std::vector<int> a(N, 1);
    std::vector<int> b(N, 1);

    if (size > 4 || size == 3) {
        std::cout << "Need 1, 2, or 4 processes for this program."
                  << std::endl;
        return 1;
    }

    int process_elements = N / size;
    int starting_point   = rank * process_elements;
    int ending_point     = starting_point + process_elements;

    double start_time = MPI_Wtime();

    int result = scalar_product(a, b, starting_point, ending_point);

    int step = 1;

    while (step < size) {
        if (rank % (2 * step) == 0) {
            int received_value;
            int partner = rank + step;
            if (partner < size) {
                MPI_Recv(&received_value, 1, MPI_INT, partner,
                         MPI_ANY_TAG, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                result += received_value;
            }
        } else if (rank % (2 * step) == step) {
            int partner = rank - step;
            MPI_Send(&result, 1, MPI_INT, partner, 0, MPI_COMM_WORLD);
            break;
        }
        step *= 2;
    }

    if (rank == 0) {
        cout << "============================" << endl;
        cout << "Final dot product: " << result << endl;
        double end_time      = MPI_Wtime();
        double local_elapsed = end_time - start_time;
        cout << "Time elapsed: " << local_elapsed << endl;
```

```
        cout << "============================" << endl;
    }

    cout << "Result␣of␣scalar␣product:␣" << result << endl;
    cout << "Number␣of␣processes:␣"      << size   << endl;
    cout << "Each␣element␣got:␣"         << process_elements
         << "␣elements."                 << endl;

    MPI_Finalize();
    return 0;
}
```

Results

All vectors are 1s; expected dot product = N. Times are wall-clock seconds.

P	N	Result	Time (s)
1	1024	1024	0.000451
1	2048	2048	0.0000834
1	4096	4096	0.0000785
2	1024	1024	0.0000694
2	2048	2048	0.0000808
2	4096	4096	0.0000713
4	1024	1024	0.0000703
4	2048	2048	0.0000642
4	4096	4096	0.000254

Table 2: Dot product correctness and timing results (hand-coded)

Problem 4: Dot Product — LLM Vibe Coding Comparison

LLM Environment

Attribute	**Value**
Model	Claude Sonnet 4.6 (`claude-sonnet-4-6`)
Provider	Anthropic
Interface	claude.ai web chat
Session Date	February 20, 2026

Prompts & Responses

Prompt 1:

> *"Write an MPI C program that computes the dot product of two vectors of length N using P processes. Requirements: divide data as equally as possible among processes (handle N not divisible by P); each process computes its local partial sum via element-wise multiplication; use only* `MPI_Send` *and* `MPI_Recv` *(no collectives like* `MPI_Reduce`*);*

implement your own binary tree reduction; use `MPI_Wtime()` to time only the computation and reduction phase; use easy test data so correctness can be verified (not all zeros); accept N as a command-line argument; print result and time from rank 0."

Response: Claude produced a complete, correct program on the first prompt. Test data: `a[i] = offset + i + 1`, `b[i] = 1.0`; expected result $= N(N+1)/2$, printed alongside the computed value for verification.

```c
/* dotprod_llm.c  --  LLM-generated (Claude Sonnet 4.6), Problem 4
 * Compile: mpicc -O2 -o dotprod_llm dotprod_llm.c -lm
 * Run:     mpirun -np <P> ./dotprod_llm <N>
 */
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(int argc, char *argv[]) {
    int rank, size, N;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc < 2) {
        if (rank == 0) fprintf(stderr, "Usage: %s <N>\n", argv[0]);
        MPI_Finalize(); return 1;
    }
    N = atoi(argv[1]);

    /* base elements each; first (N%P) ranks get one extra */
    int base      = N / size;
    int remainder = N % size;
    int local_n   = (rank < remainder) ? base + 1 : base;
    int offset    = rank * base + (rank < remainder ? rank : remainder);

    double *a = (double *)malloc(local_n * sizeof(double));
    double *b = (double *)malloc(local_n * sizeof(double));
    if (!a || !b) {
        fprintf(stderr, "malloc failed\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    /* a[i] = global_index+1, b[i] = 1  =>  dot product = N*(N+1)/2 */
    for (int i = 0; i < local_n; i++) {
        a[i] = (double)(offset + i + 1);
        b[i] = 1.0;
    }

    MPI_Barrier(MPI_COMM_WORLD);
    double t_start = MPI_Wtime();

    double local_sum = 0.0;
    for (int i = 0; i < local_n; i++)
        local_sum += a[i] * b[i];
```

```

    /* Tree reduction: step doubles each round */
    double received;
    for (int step = 1; step < size; step *= 2) {
        if (rank % (2 * step) == 0) {
            int partner = rank + step;
            if (partner < size) {
                MPI_Recv(&received, 1, MPI_DOUBLE, partner, 0,
                         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
                local_sum += received;
            }
        } else if (rank % (2 * step) == step) {
            MPI_Send(&local_sum, 1, MPI_DOUBLE, rank - step, 0,
                     MPI_COMM_WORLD);
            break;
        }
    }

    double t_end = MPI_Wtime();

    if (rank == 0) {
        double expected = (double)N * (N + 1) / 2.0;
        printf("N=%d P=%d | dot=%.0f expected=%.0f %s | time=%.6f s\n",
               N, size, local_sum, expected,
               fabs(local_sum - expected) < 0.5 ? "CORRECT" : "WRONG",
               t_end - t_start);
    }
    free(a); free(b);
    MPI_Finalize();
    return 0;
}
```

Comparison & Analysis

The tree reduction structure is identical in both versions. Key differences:

- **Uneven partitions:** The LLM correctly handles `N % P != 0` via `base + remainder` logic. The hand-coded version assumes even division.
- **Data type:** The LLM used `double`; the hand-coded version used `int`, which risks overflow at large N.
- **Timing:** The LLM added `MPI_Barrier` before `MPI_Wtime()` to synchronize the start point across all processes.
- **Verification:** The LLM prints expected vs. computed result automatically.

Both produce correct output for the tested cases.

Problem 5: Communicator Splitting with `MPI_Comm_split`

Approach

`MPI_Comm_split` groups processes by color value. Using `rank/Q` as color groups processes into rows; `rank%Q` groups them into columns. Run with $P = 2$, $Q = 3$ (6 processes total). Each process prints its world rank alongside its rank within each sub-communicator.

Code

```cpp
#include <iostream>
#include <mpi.h>

int main(int argc, char** argv) {

    MPI_Init(&argc, &argv);
    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        std::cout << "This program requires at least two processes."
                  << std::endl;
        MPI_Finalize();
        return 1;
    }

    int Q = 3;
    int color1 = rank / Q;  // same color = same row
    MPI_Comm row_comm;
    MPI_Comm_split(MPI_COMM_WORLD, color1, rank, &row_comm);

    int color2 = rank % Q;  // same color = same column
    MPI_Comm col_comm;
    MPI_Comm_split(MPI_COMM_WORLD, color2, rank, &col_comm);

    int row_rank, col_rank;
    MPI_Comm_rank(row_comm, &row_rank);
    MPI_Comm_rank(col_comm, &col_rank);

    std::cout << "World rank " << rank
              << " | row " << color1 << " rank " << row_rank
              << " | col " << color2 << " rank " << col_rank
              << std::endl;

    MPI_Comm_free(&row_comm);
    MPI_Comm_free(&col_comm);
    MPI_Finalize();
}
```

Results

Run with `mpirun -n 6`. Output order is nondeterministic:

```
World rank 2 | row 0 rank 2 | col 2 rank 0
World rank 3 | row 1 rank 0 | col 0 rank 1
World rank 4 | row 1 rank 1 | col 1 rank 1
World rank 5 | row 1 rank 2 | col 2 rank 1
World rank 0 | row 0 rank 0 | col 0 rank 0
World rank 1 | row 0 rank 1 | col 1 rank 0
```

Ranks 0–2 share row 0; ranks 3–5 share row 1. Ranks $\{0,3\}$, $\{1,4\}$, $\{2,5\}$ share columns 0, 1, 2 respectively. In-group ranks are correct in all cases.

Problem 6: MPI Collective Operations

Three parts. The first two show how to build a higher-level collective out of lower-level ones; the third just uses extttMPI_Alltoall directly to send personalized data between every pair of processes.

Part 1: Replicating `MPI_Allreduce` with `MPI_Reduce` + `MPI_Bcast`

`MPI_Reduce` collects all values to rank 0 and computes the sum; `MPI_Bcast` distributes the result to all processes. This replicates `MPI_Allreduce`. Each process contributes its rank; expected sum $= 0 + 1 + 2 + 3 = 6$.

```cpp
#include <iostream>
#include <mpi.h>

using namespace std;

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size, value, result;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    value = rank;
    result = 0;

    if (size < 4) {
        std::cout << "At least four processes are needed for this program."
                  << std::endl;
        MPI_Finalize();
        return 1;
    }

    // Reduce all values to rank 0
    MPI_Reduce(&value, &result, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

    // Broadcast the result to all processes
    MPI_Bcast(&result, 1, MPI_INT, 0, MPI_COMM_WORLD);

    std::cout << "Rank " << rank
              << " has the result of the reduction: " << result
```

```
                << std::endl;

    MPI_Finalize();
    return 0;
}
```

Output:

```
Rank 0 has the result of the reduction: 6
Rank 1 has the result of the reduction: 6
Rank 2 has the result of the reduction: 6
Rank 3 has the result of the reduction: 6
```

Now the same thing using `MPI_Allreduce` directly:

```
#include <iostream>
#include <mpi.h>

using namespace std;

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size, value, result;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    value = rank;
    result = 0;

    if (size < 4) {
        std::cout << "At least four processes are needed for this program."
                  << std::endl;
        MPI_Finalize();
        return 1;
    }

    MPI_Allreduce(&value, &result, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);

    std::cout << "Rank " << rank
              << " has the result of the reduction: " << result
              << std::endl;

    MPI_Finalize();
    return 0;
}
```

Output:

```
Rank 0 has the result of the reduction: 6
Rank 1 has the result of the reduction: 6
Rank 2 has the result of the reduction: 6
Rank 3 has the result of the reduction: 6
```

Both produce the correct sum. The manual version requires two collective operations where `MPI_Allreduce` requires one.

Part 2: Replicating `MPI_Allgather` with `MPI_Gather` + `MPI_Bcast`

`MPI_Gather` collects one value per process onto rank 0; `MPI_Bcast` distributes the full array to all processes. This replicates `MPI_Allgather`.

```
#include <iostream>
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size, value;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    value = rank;
    int recvbuff[size];

    if (size < 4) {
        std::cout << "At least four processes are needed for this program."
                  << std::endl;
        MPI_Finalize();
        return 1;
    }

    MPI_Gather(&value, 1, MPI_INT, recvbuff, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Bcast(recvbuff, size, MPI_INT, 0, MPI_COMM_WORLD);

    std::cout << "Values collected on process: " << rank << " "
              << recvbuff[0] << recvbuff[1] << recvbuff[2]
              << std::endl;

    MPI_Finalize();
    return 0;
}
```

Output:

```
Values collected on process: 0 012
Values collected on process: 1 012
Values collected on process: 2 012
Values collected on process: 3 012
```

Now the same using `MPI_Allgather` directly:

```
#include <iostream>
#include <mpi.h>

int main(int argc, char** argv) {
```

```
    MPI_Init(&argc, &argv);

    int rank, size, value;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    value = rank;
    int recvbuff[size];

    if (size < 4) {
        std::cout << "At least four processes are needed for this program."
                  << std::endl;
        MPI_Finalize();
        return 1;
    }

    MPI_Allgather(&value, 1, MPI_INT, &recvbuff, 1, MPI_INT, MPI_COMM_WORLD);

    std::cout << "Values collected on process: " << rank << " "
              << recvbuff[0] << recvbuff[1] << recvbuff[2]
              << std::endl;

    MPI_Finalize();
    return 0;
}
```

Output:

```
Values collected on process: 0 012
Values collected on process: 1 012
Values collected on process: 2 012
Values collected on process: 3 012
```

Both give every process the full array `[0, 1, 2, 3]`. The `012` formatting is due to missing spaces in the print statement; the data is correct.

Part 3: Personalized Communication with `MPI_Alltoall`

`MPI_Alltoall` sends element j of each process's send buffer to process j, with each process sending a different value to each destination. Each process r sets `sendbuff[i]` $= r \times 300 + i \times 100$.

```
#include <iostream>
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int recvbuff[size];
```

```
    int sendbuff[size];

    if (size < 4) {
        std::cout << "At least four processes are needed for this program."
                  << std::endl;
        MPI_Finalize();
        return 1;
    }

    for (int i = 0; i < size; i++) {
        sendbuff[i] = rank * 300 + i * 100;
    }

    std::cout << "Starting values on process: " << rank << " "
              << sendbuff[0] << " " << sendbuff[1] << " "
              << sendbuff[2] << std::endl;

    MPI_Alltoall(&sendbuff, 1, MPI_INT, recvbuff, 1, MPI_INT, MPI_COMM_WORLD);

    std::cout << "Values collected on process: " << rank << " "
              << recvbuff[0] << " " << recvbuff[1] << " "
              << recvbuff[2] << std::endl;

    MPI_Finalize();
    return 0;
}
```

Output:

```
Starting values on process: 0 0 100 200
Starting values on process: 1 300 400 500
Starting values on process: 2 600 700 800
Starting values on process: 3 900 1000 1100
Values collected on process: 0 0 300 600
Values collected on process: 1 100 400 700
Values collected on process: 2 200 500 800
Values collected on process: 3 300 600 900
```

Each process receives element r from every sender, producing a column of the original send matrix. Process 0 receives $\{0, 300, 600, 900\}$; process 1 receives $\{100, 400, 700, 1000\}$, etc. Output is correct.

Problem G1: Conway's Game of Life with 2D MPI Decomposition

Approach

The grid is $N \times N$ and divided among $P \times Q$ processes arranged in a 2D layout. Each process owns a `local_rows` $\times$ `local_cols` subgrid where `local_rows` $= N/P$ and `local_cols` $= N/Q$. Each subgrid is padded with a one-cell ghost (halo) border, making the allocated array (`local_rows` $+ 2) \times$ (`local_cols` $+ 2$).

Neighbor ranks are computed using toroidal (wrap-around) arithmetic for all 8 directions:

$$
\begin{aligned}
\texttt{up} &= ((\texttt{row} - 1 + P) \bmod P) \times Q + \texttt{col} \\
\texttt{down} &= ((\texttt{row} + 1) \bmod P) \times Q + \texttt{col} \\
\texttt{left} &= \texttt{row} \times Q + (\texttt{col} - 1 + Q) \bmod Q \\
\texttt{right} &= \texttt{row} \times Q + (\texttt{col} + 1) \bmod Q
\end{aligned}
$$

Corner neighbors follow the same pattern. This ensures the simulation wraps correctly on all edges.

Each iteration proceeds as follows: halo rows are exchanged first using `MPI_Sendrecv` (top/bottom), then columns are packed into temporary buffers and exchanged (left/right), then the four corner cells are exchanged individually. Once all ghost cells are populated, the Life rules are applied to every interior cell using the full 8-neighbor count. Results are written into a second grid (`local_grid_next`), then the two grids are swapped with `std::swap`.

Double buffering avoids reading and writing the same grid simultaneously. Cells use `char`: `'*'` for alive, `' '` for dead. Initial state is randomized per process using `srand(rank)`.

After each swap, `print_full_grid` gathers all subgrids to rank 0 via `MPI_Gather`, reconstructs the full $N \times N$ board, and prints it. `MPI_Barrier` follows to keep all processes in sync before the next iteration.

Code

```cpp
#include <iostream>
#include <vector>
#include <mpi.h>
#include <unistd.h>


/*
 * print_full_grid() -- LLM-generated (Claude Sonnet 4.6, Anthropic)
 * Generated via claude.ai on 2026-03-04
 * Prompt: "I have some code for a game of life simulation, but I need a fancy
 *          function that will print out what is happening inside so it can be
 *          watched. Generate that for this code."
 * Model output used with minor integration edits (parameter passing, call site).
 */
void print_full_grid(char** local_grid, int local_rows, int local_cols,
                     int N, int P, int Q, int rank, int row, int col, int iter) {

    // Pack interior cells (no ghost rows/cols) into a flat send buffer
    std::vector<char> send_buf(local_rows * local_cols);
    for (int i = 0; i < local_rows; i++)
        for (int j = 0; j < local_cols; j++)
            send_buf[i * local_cols + j] = local_grid[i + 1][j + 1];

    // Rank 0 allocates a receive buffer for all processes' data
```

```
    std::vector<char> recv_buf;
    if (rank == 0)
        recv_buf.resize(N * N);  // Total cells across all processes

    MPI_Gather(
        send_buf.data(), local_rows * local_cols, MPI_CHAR,
        recv_buf.data(), local_rows * local_cols, MPI_CHAR,
        0, MPI_COMM_WORLD
    );

    if (rank == 0) {
        // Reconstruct the full NxN grid from gathered blocks.
        // MPI_Gather deposits process r's block at recv_buf[r * local_rows * local_cols].
        // Each process r occupies grid row-block (r/Q) and col-block (r%Q).
        std::vector<std::vector<char>> full(N, std::vector<char>(N, '␣'));

        for (int r = 0; r < P * Q; r++) {
            int proc_row = r / Q;   // which row-band this process owns
            int proc_col = r % Q;   // which col-band this process owns
            int row_offset = proc_row * local_rows;
            int col_offset = proc_col * local_cols;

            for (int i = 0; i < local_rows; i++)
                for (int j = 0; j < local_cols; j++)
                    full[row_offset + i][col_offset + j] =
                        recv_buf[r * local_rows * local_cols + i * local_cols + j];
        }

        // Clear terminal and print
        std::cout << "\033[2J\033[H";  // ANSI: clear screen, cursor to top-left
        std::cout << "   ";
        for (int j = 0; j < N; j++) std::cout << "   ";
        std::cout << "   \n";
        for (int i = 0; i < N; i++) {
            std::cout << "   ";
            for (int j = 0; j < N; j++)
                std::cout << full[i][j];
            std::cout << "   \n";
        }
        std::cout << "   ";
        for (int j = 0; j < N; j++) std::cout << "   ";
        std::cout << "   \n";
        std::cout << "␣Iteration:␣" << iter + 1
                  << "␣␣Grid:␣" << N << "x" << N
                  << "␣␣Processes:␣" << P << "x" << Q << "\n";
        std::cout.flush();
        usleep(150000);  // 150ms pause so the animation is watchable
    }

```

```
    // All processes must wait here -- rank 0 can't start the next iteration's
    // gather until everyone has finished computing their local_grid_next swap.
    MPI_Barrier(MPI_COMM_WORLD);
}

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc < 5) {
        if (rank == 0)
            std::cout << "Usage: mpirun -n <procs> ./hw_1_7 P Q N iterations"
                      << std::endl;
        MPI_Finalize(); return 1;
    }

    int P = atoi(argv[1]);
    int Q = atoi(argv[2]);
    int N = atoi(argv[3]);
    int iterations = atoi(argv[4]);

    if (P <= 0 || Q <= 0) {
        if (rank == 0)
            std::cout << "P and Q must be positive integers." << std::endl;
        MPI_Finalize(); return 1;
    }
    if (P * Q != size) {
        if (rank == 0)
            std::cout << "The product of P and Q must equal the number of processes."
                      << std::endl;
        MPI_Finalize(); return 1;
    }
    if (N % P != 0 || N % Q != 0) {
        if (rank == 0)
            std::cout << "N must be evenly divisible by both P and Q." << std::endl;
        MPI_Finalize(); return 1;
    }

    int row = rank / Q;
    int col = rank % Q;

    int up         = ((row - 1 + P) % P) * Q + col;
    int down       = ((row + 1) % P) * Q + col;
    int left       = row * Q + (col - 1 + Q) % Q;
    int right      = row * Q + (col + 1) % Q;
```

```
    int up_left    = ((row - 1 + P) % P) * Q + (col - 1 + Q) % Q;
    int up_right   = ((row - 1 + P) % P) * Q + (col + 1) % Q;
    int down_left  = ((row + 1) % P) * Q + (col - 1 + Q) % Q;
    int down_right = ((row + 1) % P) * Q + (col + 1) % Q;

    int local_rows = N / P;
    int local_cols = N / Q;

    // Allocate local grid with ghost border
    char** local_grid = new char*[local_rows + 2];
    for (int i = 0; i < local_rows + 2; ++i)
        local_grid[i] = new char[local_cols + 2];
    for (int i = 0; i < local_rows + 2; i++)
        for (int j = 0; j < local_cols + 2; j++)
            local_grid[i][j] = '␣';

    srand(rank);  // Different seed per process for varied initial state
    for (int i = 1; i <= local_rows; i++)
        for (int j = 1; j <= local_cols; j++)
            local_grid[i][j] = (rand() % 2) ? '*' : '␣';

    // Double buffer
    char** local_grid_next = new char*[local_rows + 2];
    for (int i = 0; i < local_rows + 2; ++i)
        local_grid_next[i] = new char[local_cols + 2];
    for (int i = 0; i < local_rows + 2; i++)
        for (int j = 0; j < local_cols + 2; j++)
            local_grid_next[i][j] = '␣';

    MPI_Comm row_comm, col_comm;
    MPI_Comm_split(MPI_COMM_WORLD, row, col, &row_comm);
    MPI_Comm_split(MPI_COMM_WORLD, col, row, &col_comm);

    // Columns are not contiguous in memory; pack into temporary buffers
    std::vector<char> send_buf(local_rows + 2);
    std::vector<char> recv_buf(local_rows + 2);

    for (int iter = 0; iter < iterations; iter++) {

        // Halo exchange: top row -> down neighbor's bottom ghost row
        MPI_Sendrecv(
            &local_grid[1][0], local_cols+2, MPI_CHAR, up, 0,
            &local_grid[local_rows+1][0], local_cols+2, MPI_CHAR, down, 0,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        // Halo exchange: bottom row -> up neighbor's top ghost row
        MPI_Sendrecv(
            &local_grid[local_rows][0], local_cols+2, MPI_CHAR, down, 0,
            &local_grid[0][0], local_cols+2, MPI_CHAR, up, 0,
            MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        // Halo exchange: left column
        for (int i = 0; i < local_rows+2; i++) send_buf[i] = local_grid[i
            ][1];
```

```
        MPI_Sendrecv(send_buf.data(), local_rows+2, MPI_CHAR, left, 0,
                     recv_buf.data(), local_rows+2, MPI_CHAR, right, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        for (int i = 0; i < local_rows+2; i++) local_grid[i][local_cols+1] = recv_buf[i];

        // Halo exchange: right column
        for (int i = 0; i < local_rows+2; i++) send_buf[i] = local_grid[i][local_cols];
        MPI_Sendrecv(send_buf.data(), local_rows+2, MPI_CHAR, right, 0,
                     recv_buf.data(), local_rows+2, MPI_CHAR, left, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        for (int i = 0; i < local_rows+2; i++) local_grid[i][0] = recv_buf[i];

        // Halo exchange: corners
        MPI_Sendrecv(&local_grid[local_rows][local_cols], 1, MPI_CHAR, down_right, 0,
                     &local_grid[0][0], 1, MPI_CHAR, up_left, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&local_grid[local_rows][1], 1, MPI_CHAR, down_left, 0,
                     &local_grid[0][local_cols+1], 1, MPI_CHAR, up_right, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&local_grid[1][1], 1, MPI_CHAR, up_left, 0,
                     &local_grid[local_rows+1][local_cols+1], 1, MPI_CHAR, down_right, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&local_grid[1][local_cols], 1, MPI_CHAR, up_right, 0,
                     &local_grid[local_rows+1][0], 1, MPI_CHAR, down_left, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        // Apply Life rules
        for (int i = 1; i <= local_rows; i++) {
            for (int j = 1; j <= local_cols; j++) {
                int neighbors = 0;
                for (int di = -1; di <= 1; di++)
                    for (int dj = -1; dj <= 1; dj++)
                        if (!(di == 0 && dj == 0))
                            neighbors += (local_grid[i+di][j+dj] == '*') ? 1 : 0;
                if (local_grid[i][j] == '*')
                    local_grid_next[i][j] = (neighbors < 2 || neighbors > 3) ? '␣' : '*';
                else
                    local_grid_next[i][j] = (neighbors == 3) ? '*' : '␣';
            }
        }

        // Swap grids for next iteration
        std::swap(local_grid, local_grid_next);

```

```
        print_full_grid(local_grid, local_rows, local_cols, N, P, Q, rank,
            row, col, iter);

    } // End iterations loop

    for (int i = 0; i < local_rows + 2; ++i) {
        delete[] local_grid[i];
        delete[] local_grid_next[i];
    }
    delete[] local_grid;
    delete[] local_grid_next;

    MPI_Comm_free(&row_comm);
    MPI_Comm_free(&col_comm);
    MPI_Finalize();
    return 0;
}
```

Results

Run with `mpirun -n 4 ./hw_1_7 4 1 32 200` (4 processes, 4×1 decomposition, 32×32 grid, 200 iterations). State at iteration 200:

```

                  **         *
                  **


                               **
                              * *
                              ***
                              *
                               *
          *                  **   *
          *                  ** ***
          *                  **   *
                             *    *
   **              ***             *
   *               ***          **
      *           *  *          *
     * *          *  *          *
      *           * *
      *
```

```
   *                          ***  *
                             ***** *
                            * ** **
                            * *   *


 Iteration: 200  Grid: 32x32  Processes: 4x1
```

Stable patterns (still lifes, oscillators) are present at iteration 200, consistent with correct Life rule application.

The following runs were used to verify the remaining assignment requirements:

```
# 8 processes, 2x4 grid, N=512 (minimum required by assignment)
mpirun -n 8 ./hw_1_7 2 4 512 10

# P=1, Q=1 (no inter-process communication, local boundary wrapping only)
mpirun -n 1 ./hw_1_7 1 1 32 5
```

Both completed without error and produced correct Life behavior across all configurations.